

Infosys Springboard Virtual Internship

**“Project Name: ConceptClarity:
Scientific Terminology Explainer”**

Name: Priya Anand

Date: 07 January 2026

Milestone 1 – Web App & Search Bar

Introduction

Concept Clarity is a web-based educational platform designed to simplify complex scientific and technical terminology for students and beginners. This milestone focused on building a secure and intuitive learning environment where users can register, log in, and access beginner-friendly explanations of scientific terms through an organized search system.

- Key Highlights: Developed a secure user registration and login process with password hashing.
- Implemented RESTful APIs for user authentication and term search functionality.
- Created a responsive frontend with automatic redirection for seamless user experience.
- Established SQLite database integration with proper session management for data security.

Tech Stack:

- Frontend: HTML5, CSS3, and JavaScript for intuitive user interfaces.
- Backend: FastAPI with Pydantic models for robust API development and validation.
- Database: SQLite with SQLAlchemy ORM for efficient data storage and retrieval.
- Server & Tools: Uvicorn ASGI server for high-performance backend operations.

This milestone established a reliable foundation for future enhancements, including advanced search features, user progress tracking, and expanded content libraries. The system provides a scalable and user-friendly educational platform that prepares for future improvements like personalized learning paths and interactive study tools.

Objectives

The main objectives successfully achieved in this project phase are:

1. Secure User Authentication

- Implemented robust user registration with email and username validation
- Created secure login functionality with password hashing using bcrypt
- Added duplicate user detection to prevent multiple registrations
- Implemented credential verification with proper error messages

2. Search Functionality Foundation

- Developed API endpoint for scientific term search
- Created structured response format for term explanations
- Implemented parameter validation and sanitization
- Established the search workflow from frontend to backend

3. User Experience & Navigation

- Designed intuitive authentication pages (signup/login)
- Created a dedicated search interface for scientific terms
- Implemented seamless navigation between authentication and search pages
- Added responsive design for cross-device compatibility

4. Data Security & Validation

- Implemented password hashing for secure credential storage
- Added input validation on both frontend and backend
- Configured CORS for secure cross-origin requests
- Established secure database connections with proper session management

Code Implementation

The project has been implemented using FastAPI (Python) as the backend framework, and vanilla HTML/CSS/JavaScript for the frontend interface. Below are the key components of the implementation:

Module 1: Authentication Service

The Authentication Service module is responsible for managing secure user registration and login functionalities within the application. It uses FastAPI routes combined with Pydantic models to validate user input and ensure data integrity. User passwords are encrypted using bcrypt hashing before being stored in the database to maintain security. This module ensures that only authenticated users can access the system by verifying credentials and returning appropriate responses.

Backend: schemas.py

```
from pydantic import BaseModel

class SignupRequest(BaseModel):
    username: str
    email: str
    password: str

class LoginRequest(BaseModel):
    username: str
    password: str
```

Backend: main.py

```
from fastapi import FastAPI, Depends
```

```
from fastapi.middleware.cors import CORSMiddleware
```

```
from sqlalchemy.orm import Session
```

```
from passlib.context import CryptContext
```

```
from database import engine, SessionLocal
```

```
from models import Base, User
```

```
from schemas import SignupRequest, LoginRequest
```

```
app = FastAPI()
```

```
app.add_middleware(
```

```
CORSMiddleware,
```

```
allow_origins=["http://localhost:5500"],
```

```
allow_credentials=True,
```

```
allow_methods=["*"],
```

```
allow_headers=["*"],
```

```
)
```

```
Base.metadata.create_all(bind=engine)
```

```
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
```

```
def get_db():
```

```
db = SessionLocal()
```

```
try:
```

```
yield db
```

finally:

db.close()

@app.post("/signup")

def signup(data: SignupRequest, db: Session = Depends(get_db)):

existing = db.query(User).filter(

(User.username == data.username) |

(User.email == data.email)

).first()

if existing:

return {"message": "User already exists"}

user = User(

username=data.username,

email=data.email,

password=pwd_context.hash(data.password)

)

db.add(user)

db.commit()

return {"message": "User registered successfully"}

@app.post("/login")

def login(data: LoginRequest, db: Session = Depends(get_db)):

user = db.query(User).filter(User.username == data.username).first()

if not user or not pwd_context.verify(data.password, user.password):

```
return {"message": "Invalid credentials"}
```

```
return {"message": "Login successful"}
```

```
@app.get("/search")
```

```
def search(term: str):
```

```
    return {
```

```
        "term": term,
```

```
        "explanation": f"{term} is explained in simple language for beginners."
```

```
    }
```

Module 2: Database Layer

The Database Layer module manages all user-related data using SQLAlchemy ORM. It defines the database schema and handles the creation of tables required for storing user credentials. This module is responsible for establishing database connections, managing sessions, and ensuring reliable data persistence. Secure storage and retrieval of user information is achieved through structured ORM-based operations.

Backend: database.py

```
from sqlalchemy import create_engine
```

```
from sqlalchemy.orm import sessionmaker, declarative_base
```

```
DATABASE_URL = "sqlite:///./users.db"
```

```
engine = create_engine(
```

```
    DATABASE_URL, connect_args={"check_same_thread": False}
```

```
)
```

```
SessionLocal = sessionmaker(bind=engine)
```

```
Base = declarative_base()
```

Backend: models.py

```
from sqlalchemy import Column, Integer, String
```

```
from database import Base
```

```
class User(Base):
```

```
__tablename__ = "users"
```

```
id = Column(Integer, primary_key=True, index=True)
```

```
username = Column(String, unique=True, index=True)
```

```
email = Column(String, unique=True)
```

```
password = Column(String)
```

Module 3: Term Search Service

The Term Search Service module provides basic functionality for retrieving simplified explanations of scientific terms. It exposes a REST API endpoint that accepts a term as input and returns an easy-to-understand explanation suitable for beginners. This module currently serves as a foundational implementation for term lookup. It is designed to be extended in future milestones with AI-driven and NLP-based explanation generation.

Backend: main.py

```
@app.get("/search")
```

```
def search(term: str):
```

```
return {
```

"term": *term*,

"explanation": f"{term} is explained in simple language for beginners."

}

Module 4: Frontend Integration

The Frontend Integration module enables users to interact with the system through a web-based interface. It includes pages for user registration, login, and scientific term search, developed using HTML, CSS, and JavaScript. The frontend communicates with the backend APIs using the Fetch API to send and receive data. Automatic page redirection and user feedback messages enhance usability and provide a smooth user experience.

Frontend: signup.html

<!DOCTYPE html>

<html>

<head>

<title>Sign Up</title>

```
<link rel="stylesheet" href="style.css">
```

</head>

<body>

Create Account

```
<input id="username" placeholder="Username">
```

```
<input id="email" placeholder="Email">
```

```
<input id="password" type="password" placeholder="Password">
```

```
<button onclick="signup()">Register</button>
```


<p id="msg"></p>

<script src="script.js"></script>

</body>

</html>

Frontend: login.html

<!DOCTYPE html>

<html>

<head>

<title>Login</title>

<link rel="stylesheet" href="style.css">

</head>

<body>

<h2>Login</h2>

<input id="username" placeholder="Username">

<input id="password" type="password" placeholder="Password">

<button onclick="login()">Login</button>

<p id="msg"></p>

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

Frontend: search.html

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Concept Clarity</title>
```

```
  <link rel="stylesheet" href="style.css">
```

```
</head>
```

```
<body>
```

```
<h2>Scientific Term Simplifier</h2>
```

```
<input id="term" placeholder="Enter scientific term">
```

```
<button onclick="searchTerm()">Search</button>
```

```
<p id="result"></p>
```

```
<script src="script.js"></script>
```

```
</body>
```

```
</html>
```

Frontend: script.js (API calls + redirect)

```
const FRONTEND_BASE = "http://localhost:5500";

const BACKEND_BASE = "http://127.0.0.1:8000";


// SIGNUP

function signup() {

  console.log("Signup clicked");


  fetch(`${BACKEND_BASE}/signup`, {

    method: "POST",

    headers: { "Content-Type": "application/json" },

    body: JSON.stringify({

      username: document.getElementById("username").value,

      email: document.getElementById("email").value,

      password: document.getElementById("password").value

    })

  })

  .then(response => response.json())

  then(data => {

    console.log("Signup response:", data);


    document.getElementById("msg").innerText =

      data.message || "Signup completed";


    // FORCE REDIRECT (ABSOLUTE URL)

    setTimeout(() => {
```

```
console.log("Redirecting to login...");  
window.location.replace(`${FRONTEND_BASE}/login.html`);  
}, 1000);  
})  
  
.catch(error => {  
  console.error("Signup error:", error);  
});  
}
```

// LOGIN

```
function login() {  
  console.log("Login clicked");  
  
  fetch(`${BACKEND_BASE}/login`, {  
    method: "POST",  
    headers: { "Content-Type": "application/json" },  
    body: JSON.stringify({  
      username: document.getElementById("username").value,  
      password: document.getElementById("password").value  
    })  
  })  
  
  .then(response => response.json())  
  .then(data => {  
    console.log("Login response:", data);  
  
    document.getElementById("msg").innerText =
```

```
data.message || "Login completed";
```

```
// FORCE REDIRECT (ABSOLUTE URL)
```

```
setTimeout() => {
```

```
    console.log("Redirecting to search...");
```

```
window.location.replace(`${FRONTEND_BASE}/search.html`);
```

```
}, 1000);
```

```
}
```

```
catch(error => {
```

```
    console.error("Login error:", error);
```

```
});
```

```
}
```

```
// SEARCH
```

```
function searchTerm() {
```

```
    const term = document.getElementById("term").value;
```

```
    fetch(`${BACKEND_BASE}/search?term=${encodeURIComponent(term)}`)
```

```
    then(response => response.json())
```

```
    .then(data => {
```

```
        document.getElementById("result").innerText =
```

```
        data.explanation;
```

```
    });
```

```
}
```

Frontend: style.css

```
body {  
  font-family: Arial, sans-serif;  
  margin: 50px;  
}
```

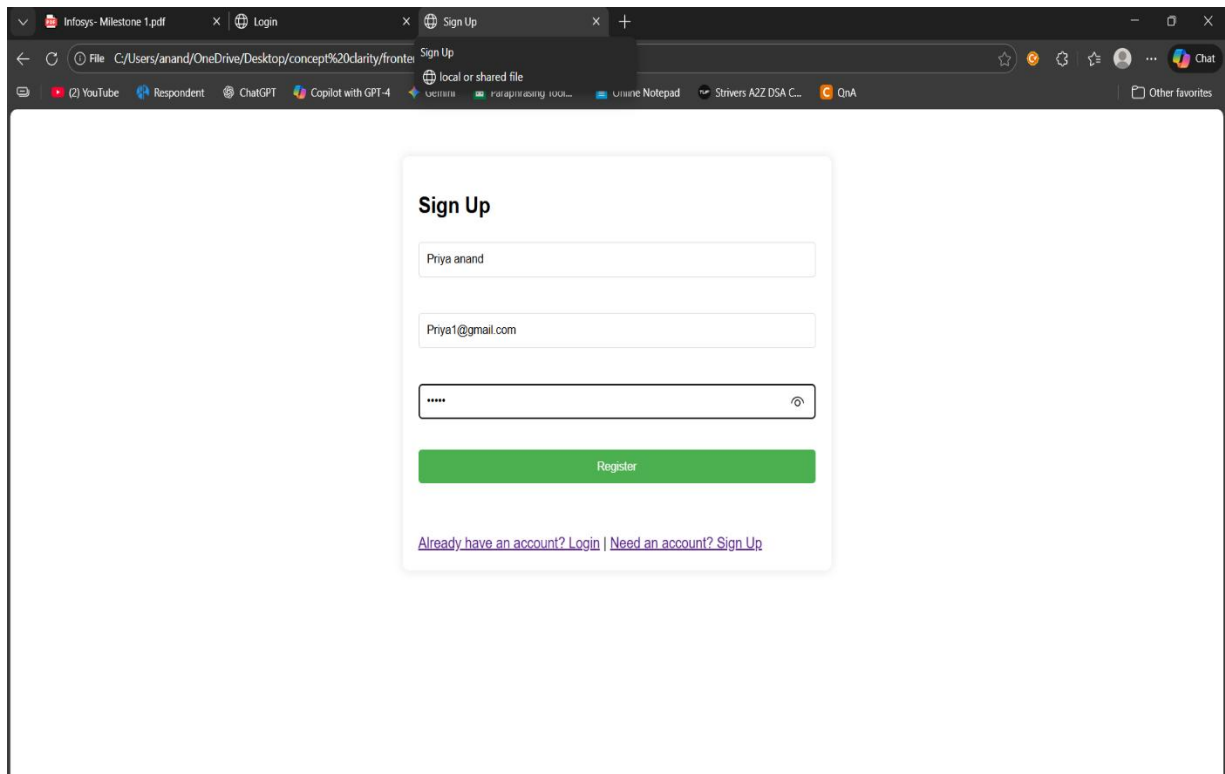
```
input {  
  display: block;  
  margin: 10px 0;  
  padding: 8px;  
  width: 260px;  
}
```

```
button {  
  padding: 8px 14px;  
  margin-top: 10px;  
}
```

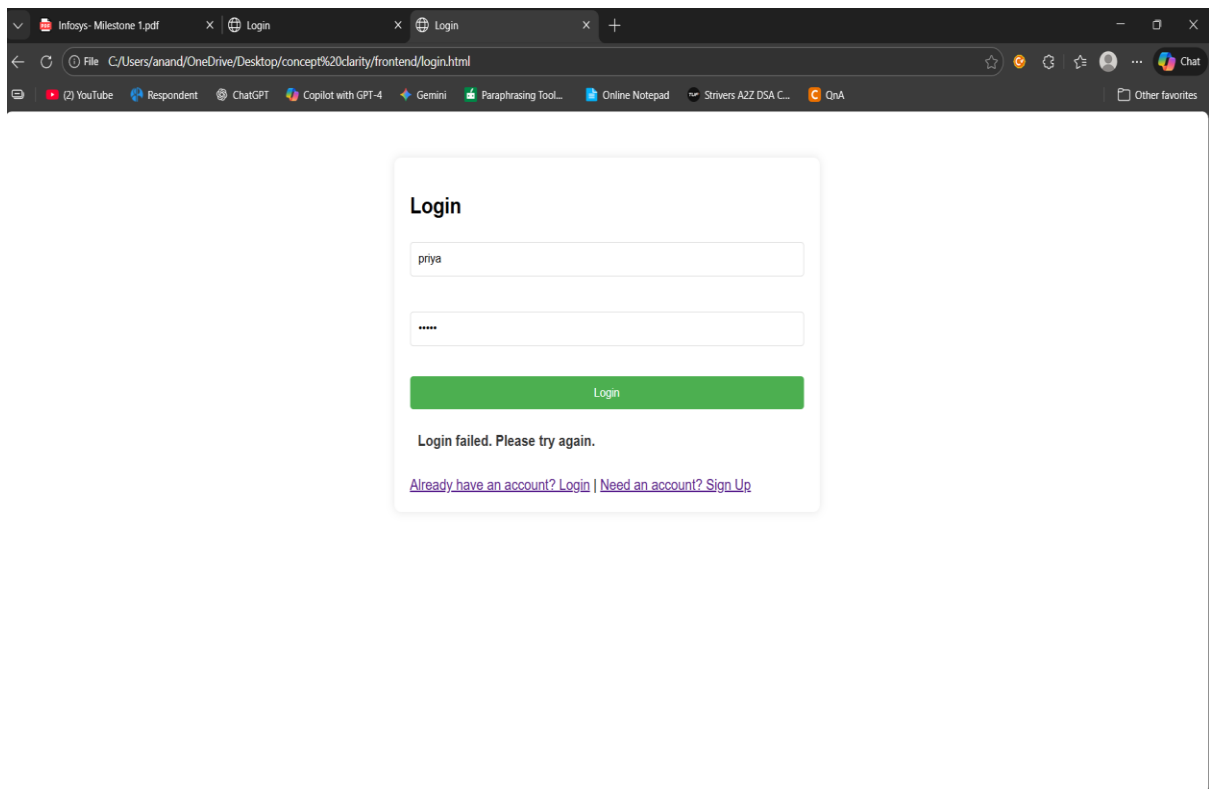
```
p {  
  margin-top: 15px;  
  font-weight: bold;  
}
```

Output screenshots:

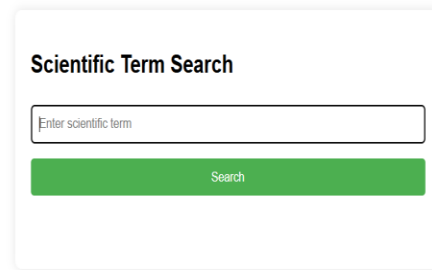
1 Sign In page:



2. Login page:



3. scientific term search page:

A screenshot of a web form titled "Scientific Term Search". It features a text input field with the placeholder text "Enter scientific term" and a green "Search" button below it.

Scientific Term Search

Search

Conclusion

In this milestone, the Concept Clarity platform successfully establishes a strong and reliable foundation for the overall application, delivering a fully functional and secure user experience from end to end. Secure user authentication has been implemented, featuring both user registration and login workflows, to ensure that only valid and verified users can access the system's core functionality. The integration of a relational database provides safe, structured, and efficient storage of user credentials, creating a trustworthy data layer that supports user management and future data expansion. Frontend and backend components communicate effectively through well-defined REST APIs, facilitating smooth and reliable data flow between the user interface and server-side logic. The term search functionality a core feature of the platform has been successfully implemented to demonstrate the application's purpose, providing clear and simplified explanations of scientific terminology based on user queries. Comprehensive input validation is applied at both the client and server levels, preventing erroneous or malicious data from compromising system integrity. Password encryption using industry-standard hashing algorithms ensures that sensitive user information remains protected against unauthorized access. Furthermore, the entire system architecture follows a modular and scalable design approach, allowing individual components to be developed, tested, and enhanced independently without disrupting the overall application. This modularity not only improves maintainability but also supports the seamless integration of more advanced features planned for later phases. The successful completion of this

milestone clearly validates the technical feasibility of the project, confirming that the chosen technologies and design patterns work effectively together to meet the platform's requirements. It also ensures a high degree of system stability and usability, as evidenced by rigorous testing of user flows such as registration, authentication, and term search operations. By delivering a robust and secure base version of the application, Milestone-1 effectively prepares the ground for integrating more sophisticated capabilities such as AI-powered explanations, personalized learning features, and enhanced interactivity in upcoming development phases, setting a solid trajectory toward the project's long-term vision.